AlumniConnect

Complete Start-to-End Technical Guide, System Architecture & Development Roadmap

A comprehensive walkthrough of the full-stack database application designed to connect students with verified alumni for job referrals.

---

1. What is AlumniConnect? (Explained to a 5-Year-Old 🧸)

Imagine a school where students want to get jobs at cool companies. But it is very hard to get a job unless a nice older brother or sister (an **Alumni**) who already works at that company introduces you to the boss.

Our project, **AlumniConnect**, is like a **Digital Post Office**:

- 📬 **The Student** writes a nice letter (a *Referral Request*) explaining why they are good at coding and why they want to work at a company (like Google).
- 👮 **The Admin** (the postmaster) checks the letter to make sure it is polite and has all the information. If it is good, they stamp it as **Approved**.
- 💼 **The Alumni** (the older brother/sister) logs in, sees the approved letters, and says, *"Hey, I work at Google! I can take this student's resume and hand it to my manager!"*
- 🚀 **The System** tracks the whole journey, sending updates when the student gets an interview or a job offer!

2. User Roles & Permission Flows 👥

AlumniConnect has three distinct user roles. Here is what they enter, what they see, and what they get out of the system:

User Role	What they Enter (Input)	What they See (Interface)	What they Get (Output)
Student	• Login / Registration details • Job details (Title, Company, URL, Description) • Pitch (Motivation, Skills, Resume Link)	• Dashboard with stats • List of own referral requests • Company search directory • Job request status timeline	• Referral requests submitted for approval • System notifications when request is approved/rejected/referred
Alumni	• Professional details (Company, Title, LinkedIn) • Referral details (Internal Ref ID, outcome updates)	• Alumni dashboard • Directory of approved student requests • Outcome	• Submitted referral recommendations for students • Status updates on referral progress

User Role	What they Enter (Input)	What they See (Interface)	What they Get (Output)
		updates (Interview Date, Feedback)	
Admin	• Review decisions (Approve/Reject buttons) • Rejection comments / reasons	• Admin control panel • List of unverified alumni accounts • Pending student requests & pending submissions	• Verified alumni profiles • Approved student requests & finalized company referrals

3. Database Relational Schema (8 Tables) 🗄️

The database is built on **PostgreSQL** and is comprised of 8 relational tables linked by foreign-key constraints:

```
erDiagram
    USERS |├-|| ALUMNI_PROFILES : "has profile (1:1)"
    USERS |├-o{ REFRESH_TOKENS : "owns (1:N)"
    USERS |├-o{ REFERRAL_REQUESTS : "creates (student) (1:N)"
    USERS |├-o{ REFERRAL_REQUESTS : "approves (admin) (1:N)"
    USERS |├-o{ REFERRAL_SUBMISSIONS : "submits (alumni) (1:N)"
    USERS |├-o{ REFERRAL_SUBMISSIONS : "approves (admin) (1:N)"
    USERS |├-o{ ENDORSEMENTS : "gives (1:N)"
    USERS |├-o{ NOTIFICATIONS : "receives (1:N)"

    COMPANIES |├-o{ REFERRAL_REQUESTS : "targeted by (1:N)"

    REFERRAL_REQUESTS |├-o{ REFERRAL_SUBMISSIONS : "referred by (1:N)"
    REFERRAL_REQUESTS |├-o{ ENDORSEMENTS : "receives (1:N)"

    USERS {
        int id PK
        string email UK
        string password
        string full_name
        string phone
        string role "student/alumni/admin"
        string status "pending/active/suspended/inactive"
        boolean is_email_verified
        string verification_token
        boolean is_active
        datetime last_login_at
        datetime created_at
        datetime updated_at
    }

    ALUMNI_PROFILES {
        int id PK
        int user_id FK "users.id"
        string company
        string job_title
```

```
        string linkedin_url
        int graduation_year
        text bio
        boolean is_verified
        int verified_by_id FK "users.id"
        datetime verified_at
        datetime created_at
        datetime updated_at
    }

    COMPANIES {
        int id PK
        string name UK
        string industry
        string website
        string careers_page
        string logo_url
        text description
        string headquarters
        string company_size
        string company_type
        boolean is_active
        boolean is_verified
        int request_count
        datetime created_at
        datetime updated_at
    }

    REFERRAL_REQUESTS {
        int id PK
        int student_id FK "users.id"
        int company_id FK "companies.id"
        string position_title
        text job_description
        string job_url
        string location
        text motivation
        text skills_match
        string resume_url
        string status "draft/pending/approved/rejected/fulfilled/closed"
        string priority "low/medium/high/urgent"
        datetime submitted_at
        datetime approved_at
        int approved_by_id FK "users.id"
        text rejection_reason
        datetime rejected_at
        int view_count
        int endorsement_count
```

```
        boolean is_urgent
        boolean is_featured
        datetime created_at
        datetime updated_at
    }

    REFERRAL_SUBMISSIONS {
        int id PK
        int request_id FK "referral_requests.id"
        int alumni_id FK "users.id"
        text notes
        string internal_reference_id
        int confidence_level
        string status
"pending/approved/rejected/submitted_to_company/interview_scheduled/offer_recei
ved/withdrawn"
        datetime approved_at
        int approved_by_id FK "users.id"
        text rejection_reason
        datetime rejected_at
        datetime submitted_to_company_at
        datetime interview_date
        text interview_feedback
        text outcome_notes
        datetime outcome_updated_at
        datetime created_at
        datetime updated_at
    }

    ENDORSEMENTS {
        int user_id PK, FK "users.id"
        int request_id PK, FK "referral_requests.id"
        text comment
        int strength "1-5"
        string endorsement_type "general/technical/cultural"
        datetime created_at
    }

    REFRESH_TOKENS {
        int id PK
        int user_id FK "users.id"
        string token UK
        datetime expires_at
        boolean revoked
        datetime created_at
    }

    NOTIFICATIONS {
```

```
        int id PK
        int user_id FK "users.id"
        string type "referral_status/profile_verified/endorsement_received"
        string title
        text message
        boolean is_read
        datetime created_at
    }
```

4. Architecture and File Import Order ⚙️

When building a full-stack project, you must build **bottom-up**. You cannot build routers without services, you cannot build services without repositories, and you cannot build repositories without database models.

```
flowchart TD
    Env[.env] ⟶|1. Loads configs| Config[app/core/config.py]
    Config ⟶|2. Configures database| DB[app/core/database.py]

    DB ⟶|3. Declares tables| Models[app/models/*.py]
    Models ⟶|4. Queries database| Repos[app/repositories/*.py]

    Models ⟶|5. Defines validation| Schemas[app/schemas/*.py]
    Repos -.→|6. Validates data flow| Schemas

    Repos & Schemas ⟶|7. Contains business rules| Services[app/services/*.py]

    Services ⟶|8. Adds security guards| Guards[app/dependencies/*.py]

    Guards ⟶|9. Resolves API endpoints| Routers[app/routers/*.py]

    Routers ⟶|10. Assembled in main entry point| Main[app/main.py]
```

5. The 13-Day Implementation Roadmap

Day 1: Project Setup & Environment Configurations

Create the project directory structure, initialize the package and python configurations, write dependency requirements, and setup environmental parameters.

- **Files written:** `Alumni_Project_Final/requirements.txt`, `.env`, `app/core/config.py`
- **Explanation:** Loads configuration settings using Pydantic Settings from a `.env` file. Pydantic validates data types (like ensuring Port is an integer). We use `lru_cache` so settings are instantiated once, speed-optimizing subsequent imports.

Day 2: Database Connection & Exceptions Setup

Configure the SQLAlchemy connector and setup custom HTTP errors.

- **Files written:** `Alumni_Project_Final/app/core/database.py`, `app/core/exceptions.py`
- **Explanation:** `database.py` generates the database Engine with pooling (keeps connection links active to prevent delays) and declares the transactional generator `get_db`. `exceptions.py` defines standard API exceptions (like NotFound, Conflict) so endpoints return clean error messages instead of server crashes.

Day 3: Security & Middlewares Setup

Establish security layers for passwords, rate limiters, and server HTTP response headers.

- **Files written:** `app/core/security.py`, `app/core/rate_limit.py`, `app/middleware/security_headers.py`, `app/middleware/error_handler.py`
- **Explanation:** Set up `bcrypt` password hashing contexts, mount `slowapi` rate limits (protecting endpoints from bot flooding), add XSS protection headers, and establish a global handler to map python exceptions to JSON responses.

Day 4: User Verification & Account Models

Generate database schemas for users, profiles, and refresh tokens.

- **Files written:** `app/models/user.py`, `app/models/refresh_token.py`, `app/models/alumni_profile.py`
- **Explanation:** Defines SQLAlchemy models mapping table schemas. Establishes the 1:1 relationship between User and AlumniProfile, and sets up explicit foreign keys to avoid naming collisions when referencing user records.

Day 5: Operation Models

Establish database tables representing companies, referral requests, submissions, and endorsements.

- **Files written:** `app/models/company.py`, `app/models/referral_request.py`, `app/models/referral_submission.py`, `app/models/endorsement.py`, `app/models/notification.py`
- **Explanation:** Defines models that track referral request status transitions (Draft → Pending → Approved → Fulfilled → Closed), outcomes, peer endorsement counts, and system notifications.

Day 6: Pydantic Validation Schemas

Write request/response schemas to validate raw API data payloads.

- **Files written:** `app/schemas/user.py`, `app/schemas/token.py`, `app/schemas/company.py`, `app/schemas/referral_request.py`, `app/schemas/referral_submission.py`, `app/schemas/response.py`
- **Explanation:** These schemas filter and format data. Input schemas validate constraints (e.g. checking that an email uses a valid format). Output schemas serialize data, omitting sensitive records like hashed passwords.

Day 7: Repositories (Data Access Layer)

Write generic classes that encapsulate database queries.

- **Files written:** `app/repositories/base.py`, `app/repositories/user.py`, `app/repositories/company.py`, `app/repositories/referral_request.py`, `app/repositories/refresh_token.py`
- **Explanation:** Implements a Repository Pattern. `BaseRepository` uses generics to expose common CRUD queries (find, update, delete). Subclasses contain custom SQL queries,

decoupling database queries from business services.

Day 8: Authentication Services & Guard Dependencies

Build core business workflows and route permission guards.

- **Files written:** `app/services/auth.py`, `app/dependencies/auth.py`, `app/dependencies/roles.py`
- **Explanation:** Handles password verification, access token signing, email token verification, and role-based guards (`require_student`, `require_alumni`, `require_admin`).

Day 9: Routers & FastAPI Assembly

Construct URL routes and integrate configurations into the main app object.

- **Files written:** `app/routers/auth.py`, `app/routers/referral_requests.py`, `app/routers/admin.py`, `app/main.py`
- **Explanation:** Defines URL endpoints, injects services and dependency guards, registers routes, and configures CORS middleware to allow external HTTP connections from the frontend.

Day 10: Seeding & Database Migrations

Set up Alembic migration tracking and insert test credentials.

- **Files written:** `alembic/env.py`, `alembic/versions/001_initial_schema.py`, `scripts/seed_data.py`
- **Explanation:** Standardizes database schema history and populates database rows with test credentials (admin, student, and alumni accounts) for development debugging.

Day 11: Frontend Framework, Clients & State Management

Setup Vite configurations, create an Axios client with auto token refresh, and build global stores.

- **Files written:** `Alumni_Frontend/package.json`, `src/lib/api-client.ts`, `src/store/auth.ts`, `src/types/index.ts`
- **Explanation:** Creates the Axios client with request/response interceptors to automatically request new tokens on `401 Unauthorized` responses. Sets up a Zustand store to handle user session persistence.

Day 12: Route Protection and Services

Configure path routing, add route guards, and construct frontend API utilities.

- **Files written:** `src/App.tsx`, `src/layouts/MainLayout.tsx`, `src/layouts/AuthLayout.tsx`, `src/services/auth.service.ts`, `src/services/referral-request.service.ts`
- **Explanation:** Defines the application router tree, implements a `ProtectedRoute` wrapper component to filter users by role, and writes API service objects to request backend transactions.

Day 13: UI Views & Dashboard Pages

Create visual pages and dashboard views.

- **Files written:** `src/pages/Dashboard.tsx`, `src/pages/ReferralRequests.tsx`, `src/pages/admin/AdminPanel.tsx`, `src/pages/auth/Login.tsx`
- **Explanation:** Renders visual pages: registration forms with validation (`react-hook-form` + `zod`), statistics counters, request lists, and responsive action layouts.

6. Line-by-Line Code Explanations 💻

A. Database Connection (`app/core/database.py`)

```
import urllib.parse
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from app.core.config import settings
```

- `urllib.parse` : Python library used to URL-encode special characters in the database password.
- `create_engine` : The core SQLAlchemy connector that manages connections to the PostgreSQL database.
- `declarative_base` : Generates the base database mapping class that SQL models inherit from.
- `sessionmaker` : Instantiates transactional database transactions (Sessions).
- `settings` : Configuration settings loaded from the `.env` file.

```
password = urllib.parse.quote_plus(settings.DATABASE_PASSWORD)
```

- Ensures password characters (like `@` , `#` , `/`) are encoded safely to prevent database connection URL parsing errors.

```
SQLALCHEMY_DATABASE_URL = (
    f"postgresql://{settings.DATABASE_USERNAME}:{password}@"
    f"{settings.DATABASE_HOSTNAME}:{settings.DATABASE_PORT}/"
    f"{settings.DATABASE_NAME}"
)
```

- Assembles the PostgreSQL connection string using our formatted environment parameters.

```
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
```

```
    pool_size=settings.DATABASE_POOL_SIZE,
    max_overflow=settings.DATABASE_MAX_OVERFLOW,
    pool_timeout=30,
    pool_recycle=1800,
    echo=settings.DEBUG,
)
```

- Instantiates the database connection engine.
- `pool_size=10` : Keeps 10 database connection links open.
- `max_overflow=20` : Creates up to 20 temporary connections under heavy load.
- `pool_timeout=30` : Aborts a connection request if it waits more than 30 seconds.
- `pool_recycle=1800` : Recreates connection threads every 30 minutes to prevent timeouts.
- `echo=DEBUG` : Prints raw SQL queries to the console in Debug mode.

```
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
)
```

- Creates the transaction session factory. We disable autocommit/autoflush so that edits are only saved when explicitly committed in services.

```
Base = declarative_base()
```

- Initializes the model mapper class. Classes inheriting from `Base` map to database tables.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- Database session dependency generator.

- `db = SessionLocal()` : Opens a connection.
- `yield db` : Hands the session to the calling function.
- `finally: db.close()` : Closes the connection and returns it to the connection pool when the request completes.

B. Axios Interceptor Client (`Alumni_Frontend/src/lib/api-client.ts`)

```
import axios, { AxiosError, InternalAxiosRequestConfig } from 'axios';
const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000/api/v1';

export const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});
```

- Creates our default Axios client, targeting the backend API version prefix.

```
apiClient.interceptors.request.use(
  (config: InternalAxiosRequestConfig) => {
    const token = localStorage.getItem('access_token');
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);
```

- **Request Interceptor:** Runs right before every outgoing API request.
- Retrieves the `access_token` from LocalStorage.
- Appends it to the header as `Authorization: Bearer <token>` to authenticate with backend endpoints.

```
apiClient.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as InternalAxiosRequestConfig & { _retry?:
boolean };
```

- **Response Interceptor:** Handles responses from backend endpoints. Processes errors to check if token rotation is required.

```
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
```

- If the API returns a `401 Unauthorized` (token expired) and we haven't already retried this request, set the retry flag to prevent loops.

```
      try {
        const refreshToken = localStorage.getItem('refresh_token');
        if (!refreshToken) {
          throw new Error('No refresh token');
        }
```

- Checks if a long-term refresh token is available in local storage.

```
        const response = await axios.post(`${API_BASE_URL}/auth/refresh`, {
          refresh_token: refreshToken,
        });

        const { access_token, refresh_token: new_refresh_token } =
response.data.data;

        localStorage.setItem('access_token', access_token);
        localStorage.setItem('refresh_token', new_refresh_token);
```

- Requests new tokens from the backend `/auth/refresh` endpoint using the refresh token.
- Stores the new tokens in local storage.

```
        if (originalRequest.headers) {
          originalRequest.headers.Authorization = `Bearer ${access_token}`;
        }
        return apiClient(originalRequest);
```

- Updates the expired header with the fresh token and resends the original API request, creating a seamless user experience.

```
      } catch (refreshError) {
        localStorage.removeItem('access_token');
        localStorage.removeItem('refresh_token');
        localStorage.removeItem('user');
        window.location.href = '/login';
        return Promise.reject(refreshError);
      }
    }
    return Promise.reject(error);
  }
);
```

- If token rotation fails (e.g. refresh token has expired), it clears storage keys and redirects the user to the Login screen.

7. Build and Run Instructions 🚀

A. Backend Setup (`Alumni_Project_Final`)

1. Open PowerShell and navigate to the backend folder:

```
cd Alumni_Project_Final
```

2. Create and activate a virtual environment:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

3. Install backend requirements:

```
pip install -r requirements.txt
```

4. Copy and configure environment settings:

```
cp .env.example .env
```

Open `.env` and set your database connection credentials.

5. Apply schemas and insert mock seed data:

```
alembic upgrade head
python scripts/seed_data.py
```

6. Run the development server:

```
uvicorn app.main:app --reload
```

B. Frontend Setup (`Alumni_Frontend`)

1. Open a new terminal window and go to the frontend directory:

```
cd Alumni_Frontend
```

2. Install the node packages:

```
npm install
```

3. Copy the local environment config:

```
cp .env.example .env
```

4. Launch the web app:

```
npm run dev
```

C. Default Credentials

Role	Email Address	Password
Admin	`admin@alumniconnect.com`	`Admin123`
Student	`student@example.com`	`Student123`
Alumni	`alumni@example.com`	`Alumni123`